



Universidad Nacional de La Matanza
Florencio Varela 1903 - San Justo - Buenos Aires - Argentina

Departamento de Ingeniería e Investigaciones Tecnológicas

Cátedra de Virtualización de Hardware (3654)

Jefe de Cátedra:

Alexis Villamayor

Docentes:

Fernando Boettner

Jefe de trabajos prácticos:

Ramiro de Lizarralde

Ayudantes:

Fernando Piubel

Año:

2026 – Primer cuatrimestre

Actividad Práctica de Laboratorio Bash y Powershell

Condiciones de entrega

- Se debe entregar por plataforma MIEL un archivo con formato ZIP o TAR (no se aceptan RAR u otros formatos de compresión/empaquetamiento de archivos), conteniendo la carátula que se publica en MIEL junto con los archivos de la resolución del trabajo.
- Se debe entregar el código fuente de cada uno de los ejercicios resueltos tanto en Bash como en Powershell. Si un ejercicio se resuelve en un único lenguaje se lo considerará incompleto y, por lo tanto, desaprobado.
- Se deben entregar lotes de prueba válidos para los ejercicios que reciban archivos o directorios como parámetro.
- Los archivos de código deben tener un encabezado en el que se listen los integrantes del grupo.
- Los archivos con el código de cada ejercicio y sus lotes de prueba se deben ubicar en un directorio con la siguiente estructura:
 - APL/
 - bash/
 - ejercicio1
 - ejercicio2
 - ejercicio3
 - ejercicio4
 - ejercicio5
 - powershell/
 - ejercicio1
 - ejercicio2
 - ejercicio3
 - ejercicio4
 - ejercicio5

Criterios de corrección y evaluación generales para todos los ejercicios

- Los scripts de bash muestran una ayuda con los parámetros “-h” y “--help”. Deben permitir el ingreso de parámetros en cualquier orden, y no por un orden fijo.
- Los scripts de Powershell deben mostrar una ayuda con el comando Get-Help. Ej: “Get-Help ./ejercicio1.ps1”. Deben realizar la validación de parámetros en la sección params utilizando la funcionalidad nativa de Powershell.
- Cuando haya parámetros que reciban rutas de directorios o archivos se deben aceptar tanto rutas relativas como absolutas o que contengan espacios.
- No se debe permitir la ejecución del script si al menos un parámetro obligatorio no está presente.
- Si algún comando utilizado en el script da error, este se debe manejar correctamente: detener la ejecución del script (o salvar el error en caso de ser posible) y mostrar un mensaje informando el problema de una manera amigable con el usuario, pensando que el usuario **no** tiene conocimientos informáticos.
- Si se generan archivos temporales de trabajo se deben crear en el directorio temporal /tmp; y se deben eliminar al finalizar el script, tanto en forma exitosa como por error, para no dejar archivos basura. (Ver trap en bash y try-catch-finally en powershell)
- Deseable:
 - Utilización de funciones en el código para resolver los ejercicios.

Ejercicio 1

Objetivos de aprendizaje: manejo de archivos CSV, manejo de parámetros y salida por pantalla

Realizar un script genérico que pueda leer registros de un CSV y realizar operaciones simples de filtros, suma y cuentas sobre los campos del mismo.

El script deberá leer de la primera línea del archivo el nombre de los campos, nombres que se podrán utilizar en los parámetros para las operaciones.

Se debe poder filtrar un campo por un patrón de texto. Ejemplos: Pais = "Argentina", Provincia = "San" (filtra tanto Santa Cruz, Santa Fe, San Juan y San Luis)

Se debe poder solicitar la sumatoria de un campo o la cuenta de registros (no tiene sentido un campo para la cuenta). Ejemplos: Sumar Poblacion, Contar

Ejemplos de llamada:

```
$ procesarCSV.sh -a censo.csv -f Provincia -b Cordoba -s Poblacion
```

Consideraciones:

1. La salida debe estar debidamente formateada para ser legible y que sea claro qué se buscó y la operación realizada
2. Las operaciones Contar y Sumar son excluyentes, solamente se puede solicitar una, pero sí o sí debe realizar una operación
3. El filtro es opcional, si no se pide filtrar debe actuar sobre todos los registros

Para ejemplos de CSV pueden descargar desde:

<https://github.com/datablist/sample-csv-files?tab=readme-ov-file>

Parámetros:

Parámetro bash	Parámetro PowerShell	Descripción
-a / --archivo	-archivo	Archivo de entrada en formato CSV
-f / --filtro	-filtro	Nombre del campo a utilizar para filtrar (opcional)
-b / --buscar	-buscar	Patrón a buscar en el campo filtro (opcional, requerido si se usó el parámetro filtro)
-c / --contar	-contar	Operación para Contar registros (no tiene valor)
-s / --sumar	-sumar	Nombre del campo para la operación de suma

Ejercicio 2

Objetivos de aprendizaje: Manipulación de texto, uso de expresiones regulares y herramientas estándar del sistema.

Implementar un script que lea un texto sin formato y aplique una serie de **arreglos automáticos** para adecuarlo a las convenciones del idioma español.

El script deberá:

- Recibir un archivo de texto como parámetro.

- Procesarlo línea por línea o como un todo.
- Mostrar el texto corregido por salida estándar o guardarlo en un nuevo archivo.

Reglas de normalización a implementar

El script deberá aplicar **al menos** las siguientes correcciones. Se pueden implementar más, siempre que estén correctamente documentadas.

1. Puntuación

- Asegurar que **cada párrafo finalice con un punto**, signo de cierre de interrogación (?) o exclamación (!) según corresponda.
- Eliminar espacios innecesarios antes de los signos de puntuación (.,,:;?!).
- Asegurar un único espacio después de los signos de puntuación, cuando corresponda.

2. Uso de mayúsculas

- Convertir a **mayúscula la primera letra**:
 - Del texto.
 - De cada oración después de un punto.
 - Después de signos de cierre (?, !).
- Convertir a mayúscula los pronombres personales “Yo” cuando correspondan (opcional, nivel avanzado).

3. Signos de interrogación y exclamación

- Verificar que toda pregunta tenga **signos de apertura y cierre** (¿ ?).
- Verificar que toda exclamación tenga **signos de apertura y cierre** (¡ !).

Ejemplo:

como estas? → ¿Cómo estas?

4. Espaciado y formato

- Eliminar espacios múltiples consecutivos y reemplazarlos por un solo espacio.
- Eliminar espacios al inicio y al final de cada línea.

5. Normalización de caracteres

- Unificar comillas simples o dobles.
- Reemplazar puntos suspensivos mal escritos (....) por

Parámetros:

Parámetro bash	Parámetro PowerShell	Descripción
-a / --archivo	-archivo	Archivo de entrada
-s / --salida	-salida	Archivo de salida (opcional, se muestra por pantalla si no se informa)

Ejercicio 3

Objetivos de aprendizaje: arrays asociativos, búsqueda de archivos, manejo de archivos, AWK

Desarrollar un script que identifique los archivos duplicados en un directorio (incluyendo los subdirectorios). Para esto, se considerará que un archivo está duplicado, si su nombre y tamaño son iguales, sin importar su contenido.

La salida del script debe ser un listado solo con los nombres de los archivos duplicados y en qué path fueron encontrados, por ejemplo:

```
archivo: ejercicio3.sh
directorio: /home/user/apl
directorio: /home/user/apl/final
directorio: /home/user/apl/final/final
```

Parámetros:

Parámetro bash	Parámetro PowerShell	Descripción
-d / --directorio	-directorio	Ruta del directorio a analizar.

Ejercicio 4

Objetivos de aprendizaje: procesos demonios, monitoreo de directorios, herramientas de compresión y archivado

Se pide realizar un script que ejecute como demonio, que detecte cada vez que un archivo que contenga alguna de ciertas palabras claves, registre en un archivo de log la operación que se realizó, fecha y hora y el tamaño del archivo. La búsqueda de las palabras se debe realizar sin tener en cuenta mayúsculas o minúsculas.

Tener en cuenta que, al ser un demonio, el script debe liberar la terminal una vez ejecutado, dejando al usuario la posibilidad de ejecutar nuevos comandos.

Consideraciones:

1. El script debe quedar ejecutando por sí solo en segundo plano, el usuario no debe necesitar ejecutar ningún comando adicional a la llamada del propio script para que quede ejecutando como demonio en segundo plano.
2. La solución debe utilizar un único archivo script (por cada tecnología), no se aceptan soluciones con dos o más scripts que trabajen en conjunto.
3. El script debe poder ejecutarse nuevamente para finalizar el demonio ya iniciado. Debe validar que esté en ejecución sobre el directorio correspondiente.
4. No se debe poder ejecutar más de 1 proceso demonio para un determinado directorio al mismo tiempo.
5. El monitoreo del directorio se debe hacer utilizando inotify-tools en bash y FileSystemWatcher en Powershell.
6. El proceso demonio debe ejecutar su funcionalidad para los archivos existentes en el directorio y luego quedar a la espera de los nuevos para volver a ejecutar.

Ejemplo de uso:

```
$ ./demonio.sh -d ../descargas --palabras password,account,unlam -l log.txt
```

```
$ ./demonio.sh -d ../documentos --palabras virtualizacion,cloud,storage --log ../registro
```

```
$ ./demonio.sh -d ../descargas --kill
```

```
$ ./demonio.sh -d ../documentos --kill
```

Parámetro bash	Parámetro PowerShell	Descripción
----------------	----------------------	-------------

-d / --directorío	-directorío	Ruta del directorío a monitorear
-l / --log	-log	Archivo de log
-k / --kill	-kill	Flag (switch) que se utiliza para indicar que el script debe detener el demonio previamente iniciado. Este parámetro solo se puede usar junto con -d o -directorío.

Ejercicio 5

Objetivos de aprendizaje: conexión con APIs y web services, manejo de archivos y objetos json, cache de información

Se necesita implementar un script que facilite la consulta de información relacionada a la serie Rick and Morty. El script permitirá buscar información de los personajes por su id o su nombre a través de la api <https://rickandmortyapi.com/> y pueden enviarse más de 1 id o nombre en la ejecución del script e incluso solicitar la búsqueda por ambos parámetros. Tener en cuenta que al buscar por nombre el resultado será una lista a diferencia de la búsqueda por ID.

Una vez obtenida la información, se generará un archivo caché con la información obtenida, para evitar volver a consultarlo a la api en una subsiguiente llamada, informando la ruta donde se almacena el/los archivo/s de caché.

La información de cómo obtener los datos se puede consultar en el siguiente link:

<https://rickandmortyapi.com/documentation/#character>

Por ejemplo:

<https://rickandmortyapi.com/api/character/?name=rick>

<https://rickandmortyapi.com/api/character/1>

Se mostrará por pantalla la información básica de él/los personaje/s con el siguiente formato.

Character info: (uno por cada resultado encontrado)

Id: 1

Name: Rick Sanchez

Status: Alive

Species: Human

Gender: Male

Origin: Earth (C-137)

Location: Citadel of Ricks

Episodes: 175

En caso de ingresar un id inválido o un nombre que no traiga resultados, se deberá informar al cliente un mensaje acorde. Mismo en caso de que la api retorne un error, por ejemplo que no haya conexión.

Ejemplo de uso:

```
$ ./rickandmorty.sh --id "1,2" --nombre "rick, morty"
```

```
> ./rickandmorty.ps1 -id 1,2 -nombre rick, morty
```

Parámetro bash	Parámetro Powershell	Descripción
-i / --id	-id	Id/s de los personajes a buscar.
-n / --nombre	-nombre	Nombre/s de los personajes a buscar.
-c / --clear	-clear	Limpiar el caché guardado. No se puede utilizar junto con alguno de los parámetros de búsquedas

Los parámetros al utilizar Powershell deben ser de tipo array, es decir, no es correcto que sea un string y luego utilizar una función como “.split(',')” para obtener los valores correspondientes.